

SQL et Python

Le module `sqlite3` permet de créer et interroger une base de données à partir de Python.

```
Entrée[1]: import sqlite3
```

Création d'une table

On va créer la table suivante, qui représente une version simplifiée du bulletin des élèves d'un lycée. Les attributs sont l'id de l'élève (clé primaire), le nom de l'élève, le prénom de l'élève, sa moyenne, un mot de passe et l'adresse de l'élève :

Bulletin	
<u>id</u>	
nom	
prénom	
moyenne	
motDePasse	
adresse	

```
Entrée[ ]: connexion = sqlite3.connect('maBase.db') # connexion à la base de donn
# celle-ci est créée si elle n'existe pas

# Récupération d'un curseur (objet qui envoie les requêtes et lit les
c = connexion.cursor()

# Exécution d'une commande SQL, pour créer une nouvelle table dans la
c.execute("CREATE TABLE IF NOT EXISTS "
          + "Bulletin(id INTEGER PRIMARY KEY, Nom TEXT, Prénom TEXT, m

# insertion d'un enregistrement (une ligne) dans la table
c.execute("INSERT INTO Bulletin VALUES(1,'Simpson', 'Bart', 12, 'azert

# Sauvegarde des modifications (obligatoire sinon les modifications so
connexion.commit()

c.execute("SELECT * FROM Bulletin;")
print(c.fetchall())
# la méthode fetchall renvoie le résultat de la dernière requête
# (toutes les lignes sont placées dans une liste de tuples)
# fetch signifie "aller chercher"

# Fermeture de la connexion à la base de données (pour fermer propreme
connexion.close()
```

Remarque 1 : Python et SQL ne sont pas les mêmes langages, donc ils n'utilisent pas les mêmes mots-clés.

- En Python, les types s'appellent `int` , `float` , `str` , etc.
- En SQL, les types s'appellent `INTEGER` , `REAL` , `TEXT` , etc.

Remarque 2 : On stocke ici les mots de passe en clair pour simplifier, mais dans une vraie application, on ne fait jamais ça (on stocke un mot de passe chiffré / haché).

Exercice 1 :

- 1- Remplir la table avec trois élèves supplémentaires.
- 2- Afficher le contenu de la table *Bulletin* avec la commande SQL `SELECT` .

```
Entrée[1]: # Votre programme en Python ici
```

Exercice 2 : Écrire un programme qui demande à l'utilisateur de saisir un nom de famille, puis qui affiche les informations de toutes les personnes qui portent ce nom.

```
Entrée[2]: # Votre programme ici :
```

Exercice 3 : Écrire un programme qui génère une liste de dictionnaires Python à partir des données de la table *Bulletin*.

```
Entrée[4]: # Votre programme ici :
```

Exercice 4 : Écrire un programme qui demande à l'utilisateur de saisir successivement un nom, un prénom, une moyenne, un mot de passe et une adresse, puis qui sauvegarde dans la base de données les valeurs saisies. Ce programme pourrait être accessible aux professeurs par exemple. Ajouter ensuite quelques élèves dans la base de données avec cette méthode, puis afficher à nouveau le contenu de la base complétée.

```
Entrée[3]: # Votre programme ici
```

Exercice 5 : *Faible de sécurité : l'injection SQL (usurpation d'identité)*

Le programme ci-dessous permet à un élève d'afficher les informations personnelles contenues dans son bulletin, à condition qu'il ait donné auparavant son nom et son mot de passe.

- 1- Quelle requête va être réellement effectuée si un utilisateur malveillant fournit le mot de passe suivant : ' OR 1=1; --
- 2- Que va-t-il alors se passer ?
- 3- Comment remédier à ce problème ?

```
Entrée[4]: nom = input("Nom : ")
motDePasse = input("Mot de passe : ")
connexion = sqlite3.connect('maBase.db')
c = connexion.cursor()
requête = "SELECT * FROM Bulletin WHERE nom='" + nom + "' AND motDePas
c.execute(requête)
print(c.fetchall())
connexion.close()
```

Exercice 6 : Faille de sécurité : l'injection SQL (piratage d'une base de données)

- 1- Écrire un programme qui permet à un élève de modifier son adresse à condition qu'il ait donné auparavant son nom et son mot de passe.
- 2- Comment un utilisateur pourrait-il modifier sa moyenne de façon frauduleuse ?
- 3- Comment remédier à ce problème ?

```
Entrée[5]: # Votre programme ici
```

Comment rendre plus agréable la saisie de la commande SQL qui contient des variables Python :

Il est possible de laisser dans la requête des "trous" dénotés par les caractères `?`. Ces trous sont ensuite remplacés par les valeurs se trouvant dans la liste passée en second paramètre à `.execute()` (le premier trou est remplacé par la première valeur de la liste et ainsi de suite).

Cette méthode a aussi l'avantage de prémunir contre l'injection SQL.

Exercice 7 : Analogie entre tables et classes

1) Écrire une classe `Bulletin` correspondant à la table `Bulletin`, puis créer une instance pour chaque bulletin présent dans la base de données.

2) On a l'analogie suivante entre une table d'une base de données et une classe Python :

- Table `Bulletin` ↔ classe `Bulletin`
- Ligne ↔ objet / instance
- Colonne ↔ attribut d'instance

Différence : on peut ajouter des méthodes à une classe, on ne peut pas faire cela avec une table.

Ajouter une méthode `mention` dans la classe `Bulletin`. Cette méthode devra renvoyer une chaîne de caractère, par exemple "Assez bien" si la moyenne du bulletin est comprise entre 14 et 16.

```
Entrée[6]: # Définition de votre classe Bulletin ici
```

